CloudSage — Architecture

Logical, runtime and security architecture

1. System Architecture

User/UI
↓
API / Orchestrator
↓
Context Builder
↓
Optimization Engine ← AWS Collectors ← AWS APIs
↓
AI Reasoner ← one runtime LLM provider
↓
Risk / Policy Engine
↓
Recommendation + Approval
↓
Terraform Generator → validate → plan → plan analysis → approval → apply
↓
AWS
↓
Post-change Verification → Database / Audit

2. Major Components

Component	Responsibility
API / Orchestrator	Authentication, requests, workflows and job control
AWS Collector	Discover resources and retrieve AWS metadata
Cost Collector	Retrieve Cost Explorer data and normalize costs
Metrics Collector	Retrieve CloudWatch metrics and aggregate windows/percentiles
Terraform Intelligence	Map AWS resources to Terraform state/configuration
Optimization Engine	Deterministic candidate detection and calculations
Context Builder	Create bounded, structured AI input
AI Reasoner	Interpret evidence and produce structured recommendations
Risk/Policy Engine	Apply deterministic safety and approval rules
Terraform Engine	Generate, validate and plan infrastructure changes
Verification Engine	Measure post-change health and savings
PostgreSQL	Persist resources, costs, metrics,

	recommendations and audit data
Worker/Scheduler	Run scans and long-running analysis jobs
UI	Show findings, evidence, plans, approvals and results

3. Data Flow

1. Discover AWS resources and normalize their identities.
2. Collect cost and utilization data for the relevant period.
3. Join resource metadata, cost, metrics, Terraform and operational context.
4. Run deterministic candidate rules.
5. Build a bounded AI context package.
6. Send the package to the selected LLM through a single provider adapter.
7. Validate structured AI output against schema and deterministic facts.
8. Calculate savings independently in Python.
9. Apply policy and approval requirements.
10. Generate/modify Terraform and run fmt/validate/plan.
11. Analyze the plan for unexpected impact.
12. Require approval before apply.
13. Apply through the controlled Terraform execution path.
14. Monitor the resource after the change and compare against baseline.
15. Record actual result and audit trail.

4. Runtime AI Boundary

The runtime application uses exactly one LLM provider/API key. The backend owns AWS credentials, Terraform execution and policy enforcement. The LLM receives curated context and returns structured output; it does not receive AWS secrets.

5. Resource Mapping

CloudSage must map an AWS resource ID to its Terraform representation using Terraform state, resource IDs, tags, module outputs and other deterministic identifiers. A recommendation cannot generate a change until this mapping is sufficiently reliable.

6. Security Boundary

LLM
  │ no AWS credentials
  ↓
CloudSage API
  ↓
Policy Engine
  ↓
AWS/Terraform execution role

↓
AWS

7. Scaling Architecture

The first version can run as a single FastAPI process plus PostgreSQL. Later, long-running scans move to worker processes with Redis/queueing, and deployment can move to K3s. The optimizer should use a plugin architecture so each AWS service implements discovery, metrics, cost, candidate detection, recommendation context and verification.

8. Proposed Service Plugins

- EC2Optimizer
- EBSOptimizer
- EIPOptimizer
- S3Optimizer
- RDSOptimizer
- ALBOptimizer
- ECSOptimizer
- LambdaOptimizer
- NATGatewayOptimizer